

INFORME DE ANÁLISIS TÉCNICO PROFESIONAL

Sistema de Gestión de Clientes (ProyectoLyEP2026)

Rol: Analista Programador

Tecnologías: React 19, React Router v7, Node.js, Express, Axios, Bootstrap 5

Actividad: Práctica Profesional Supervisada - Relevamiento y Deuda Técnica

1. Resumen Ejecutivo

El prototipo actual presenta una estructura funcional base con enrutamiento y autenticación simulada, pero evidencia una importante deuda técnica y vulnerabilidades críticas de seguridad. Se detectó una inconsistencia arquitectónica en la capa de datos al coexistir llamadas directas mediante fetch nativo en componentes de vista con servicios aislados en Axios, además de credenciales y contraseñas expuestas en texto plano en la interfaz. El control de acceso basado en roles (RBAC) presenta una doble fuente de verdad en localStorage susceptible a manipulación no autorizada, y las métricas del panel principal son valores fijos sin reactividad. A pesar de estos factores, el código es modular y constituye una base viable para su refactorización hacia estándares profesionales de la industria.

2. Matriz Consolidada de Hallazgos Individuales

- **Hallazgo 1 (Seguridad - Impacto Crítico):** Contraseñas y credenciales en texto plano en autorizacionesServices.js y DetalleCliente.jsx (se expone el password del cliente en la ficha). Solución: Ocultar contraseñas en UI y preparar arquitectura para tokens JWT.
- **Hallazgo 2 (Seguridad/Arquitectura - Impacto Crítico):** Vulnerabilidad en RBAC / Doble fuente de verdad. El rol se lee directamente de localStorage.getItem("role") en DetalleCliente.jsx en lugar del contexto, permitiendo evasión de permisos. Solución: Unificar validaciones exclusivamente en AutorizacionesContext.
- **Hallazgo 3 (Arquitectura - Impacto Alto):** Capa de datos fragmentada. clientesService.js usa Axios pero solo implementa crearCliente, mientras ListaClientes y DetalleCliente hacen fetch con URLs hardcodedas. Solución: Centralizar operaciones CRUD en clientesService.js con Axios y .env.
- **Hallazgo 4 (Estado/UX - Impacto Medio):** Falta de reactividad. Al crear un cliente en FormCliente, la tabla en ListaClientes no se actualiza automáticamente. Solución: Callback de actualización o sincronización de estado compartido.
- **Hallazgo 5 (Lógica/Negocio - Impacto Medio):** Métricas estáticas en Dashboard.jsx (valores fijos 10, 3, 3). Solución: Calcular totales dinámicamente consultando los servicios de datos.
- **Hallazgo 6 (Mantenibilidad - Impacto Bajo):** Código muerto en Dashboard.jsx con renderizado condicional de Login inalcanzable. Solución: Limpiar lógica redundante.
- **Hallazgo 7 (UI/UX - Impacto Medio):** Feedback al usuario deficiente (alert nativo bloqueante en Login) y validaciones incompletas en FormCliente. Solución: Componentes Alert de Bootstrap y validación completa de campos.

3. Propuestas de Mejoras para el Equipo

- **Opción A (Arquitectura):** Centralización de capa de servicios API en clientesService.js con Axios y .env.
- **Opción B (Seguridad):** Blindaje de autenticación, control de roles (RBAC) y ocultamiento de datos sensibles.
- **Opción C (Estado y Dashboard):** Métricas dinámicas en Dashboard y sincronización de estado de clientes.

4. Backlog Priorizado para Futuras Iteraciones

1. [Seguridad] Migrar autenticación mock a Backend real con JWT y contraseñas hasheadas (Sprint 2).
2. [Arquitectura] Centralizar capa de servicios API y variables de entorno (Sprint 1 - Actual).
3. [Seguridad/Estado] Unificar gestión de roles (RBAC) y ocultar datos sensibles en vistas (Sprint 1 - Actual).
4. [Estado/UX] Sincronización reactiva de lista de clientes tras creación/eliminación (Sprint 1 - Actual).
5. [Funcionalidad] Implementar métricas dinámicas con gráficos en Dashboard (Sprint 1 - Actual).
6. [Rendimiento] Paginación y búsqueda server-side con debounce en clientes (Sprint 2).
7. [Calidad] Suite de pruebas unitarias y de integración con Vitest y React Testing Library (Sprint 3).